МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования

«ВЛАДИВОСТОКСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «ВВГУ»)

Институт информационных технологий и анализа данных
Кафедра информационных технологий и систем

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ

по дисциплине «Информатика и основы программирования»

Реализация численных алгоритмов

Студент
гр. БИС-26-1 _____ С.В. Иванов

Преподаватель _____ М.В. Водяницкий

Владивосток 2026

1 Выполнение работы

1.1 Задание 1

В первом задании необходимо разработать программу поиска наибольшего общего делителя (НОД) двух целых чисел. Программа проверяет корректность введенных чисел и, если они допустимы, вычисляет НОД с использованием алгоритма Евклида (нахождением остатков от деления).

Алгоритм решения первого задания представлен на рисунке 1.

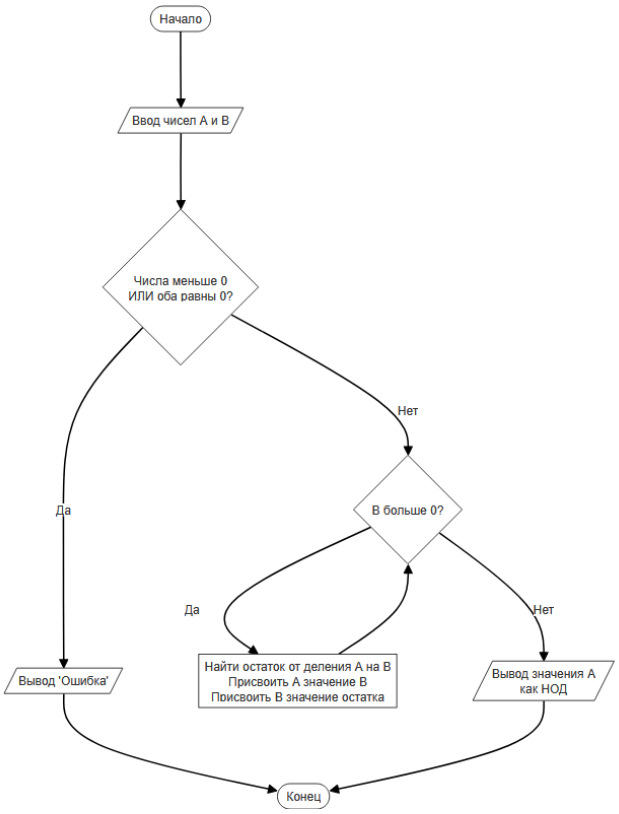


Рисунок 1 – Блок-схема программы для первого задания

Текст программы для первого задания представлен на рисунке 2.

```
1   a = int(input())
2   b = int(input())
3
4   if a < 0 or b < 0 or (a == 0 and b == 0):
5       print("Ошибка")
6   else:
7       while b > 0:
8           remainder = a % b
9           a = b
10          b = remainder
11      print(f"НОД: {a}")
```

Рисунок 2 – Текст программы для первого задания

Программа последовательно заменяет пару чисел на делитель и остаток от деления до тех пор, пока остаток не станет нулем, после чего выводит полученный НОД.

1.2 Задание 2

Во втором задании необходимо разработать программу проверки числа на простоту. Программа принимает целое число и определяет, является ли оно простым, проверяя возможные делители до квадратного корня из этого числа.

Алгоритм решения второго задания представлен на рисунке 3.

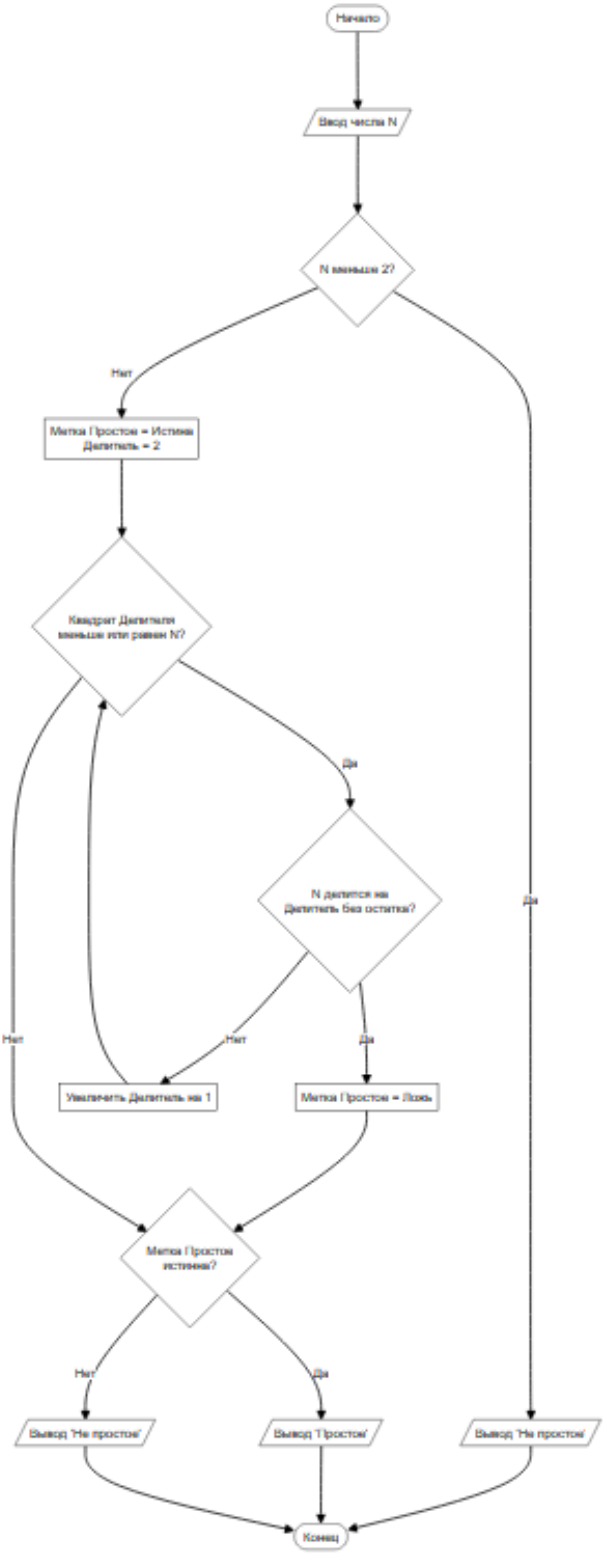


Рисунок 3 – Блок-схема программы для второго задания

Текст программы для второго задания представлен на рисунке 4.

```
n = int(input())

if n < 2:
    print("Не простое")
else:
    is_prime = True
    divisor = 2
    while divisor * divisor <= n:
        if n % divisor == 0:
            is_prime = False
            break
        divisor += 1

    if is_prime:
        print("Простое")
    else:
        print("Не простое")
```

Рисунок 4 – Текст программы для второго задания

Программа отбрасывает числа меньше двух, а для остальных перебирает делители в цикле. При нахождении делителя без остатка цикл прерывается, и программа выводит результат проверки.

1.3 Задание 3

В третьем задании необходимо разработать программу разложения числа на простые множители. Программа принимает целое число больше единицы и последовательно делит его на простые множители, выводя их на экран.

Алгоритм решения третьего задания представлен на рисунке 5.

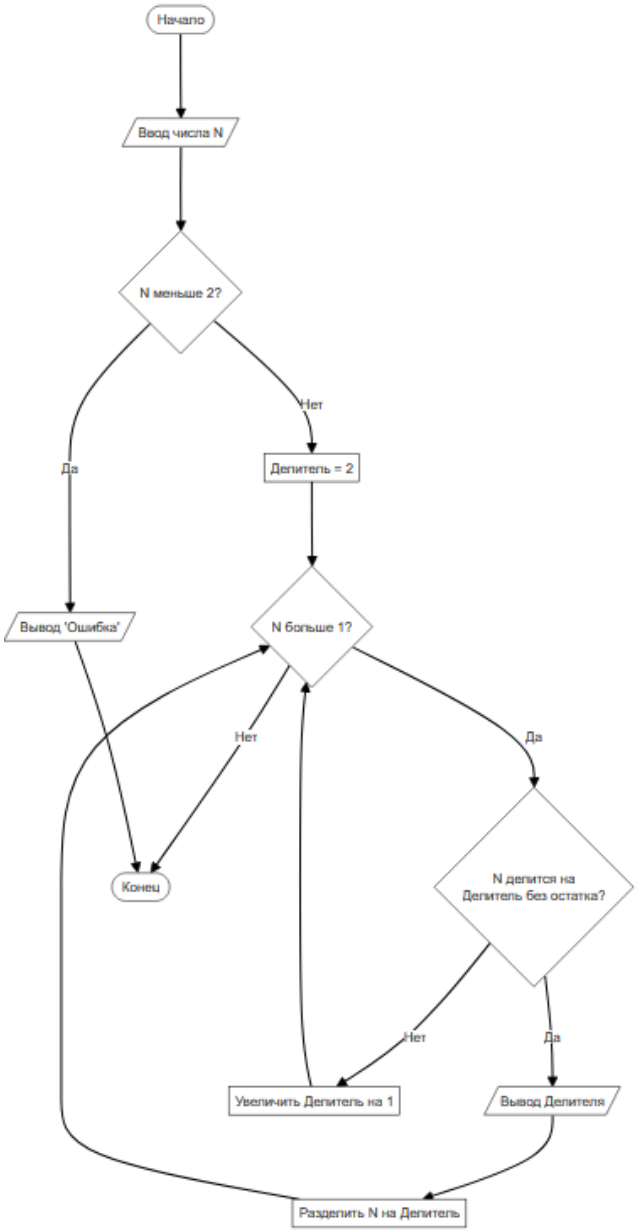


Рисунок 5 – Блок-схема программы для третьего задания

Текст программы для третьего задания представлен на рисунке 6.

```
n = int(input())

if n < 2:
    print("Ошибка")
else:
    divisor = 2
    first = True
    while n > 1:
        if n % divisor == 0:
            if not first:
                print(" ", end="")
            print(divisor, end="")
            first = False
            n = n // divisor
        else:
            divisor += 1
    print()
```

Рисунок 6 – Текст программы для третьего задания

Программа использует цикл `while`, в котором делитель проверяется на целочисленное деление с остатком. При успешном делении множитель выводится на экран, а исходное число уменьшается.

1.4 Задание 4

В четвертом задании необходимо разработать программу анализа цифр числа. Программа принимает неотрицательное целое число и вычисляет количество цифр в нем, а также сумму этих цифр с помощью математических операций.

Алгоритм решения четвертого задания представлен на рисунке 7.

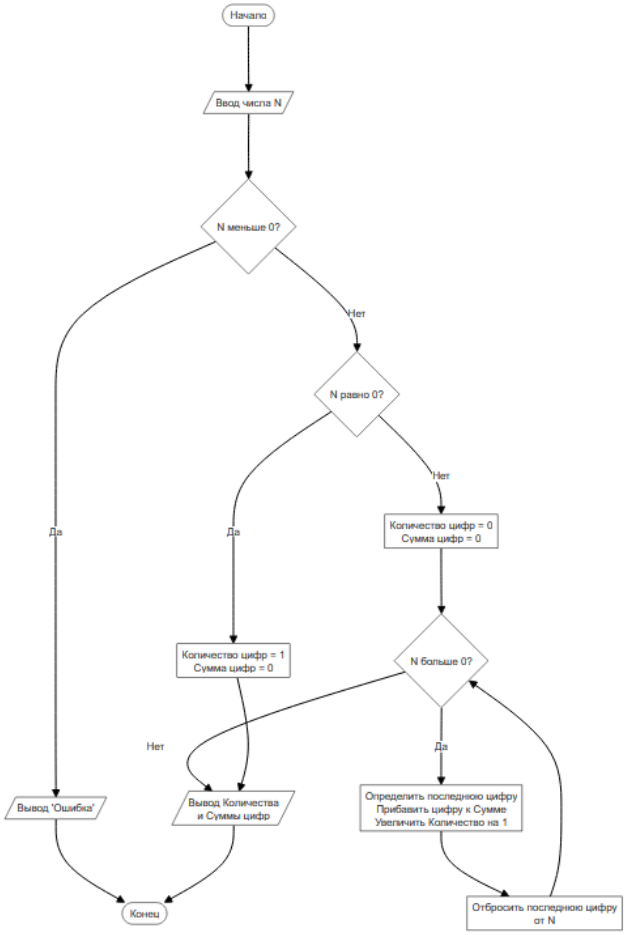


Рисунок 7 – Блок-схема программы для четвертого задания

Текст программы для четвертого задания представлен на рисунке 8.

```
n = int(input())

if n < 0:
    print("Ошибка")
else:
    if n == 0:
        count = 1
        total_sum = 0
    else:
        count = 0
        total_sum = 0
        while n > 0:
            digit = n % 10
            total_sum += digit
            count += 1
            n = n // 10

    print(f"Цифр: {count}")
    print(f"Сумма цифр: {total_sum}")
```

Рисунок 8 – Текст программы для четвертого задания

Программа последовательно извлекает младшую цифру числа остатком от деления на 10, прибавляет ее к сумме и увеличивает счетчик, после чего отбрасывает цифру целочисленным делением на 10. По завершении цикла выводятся результаты.

1.5 Задание 5

В пятом задании необходимо разработать программу построения таблицы расстояний между точками на прямой. Программа принимает количество точек от 1 до 10 и вычисляет матрицу расстояний с помощью вложенных циклов.

Алгоритм решения пятого задания представлен на рисунке 9.

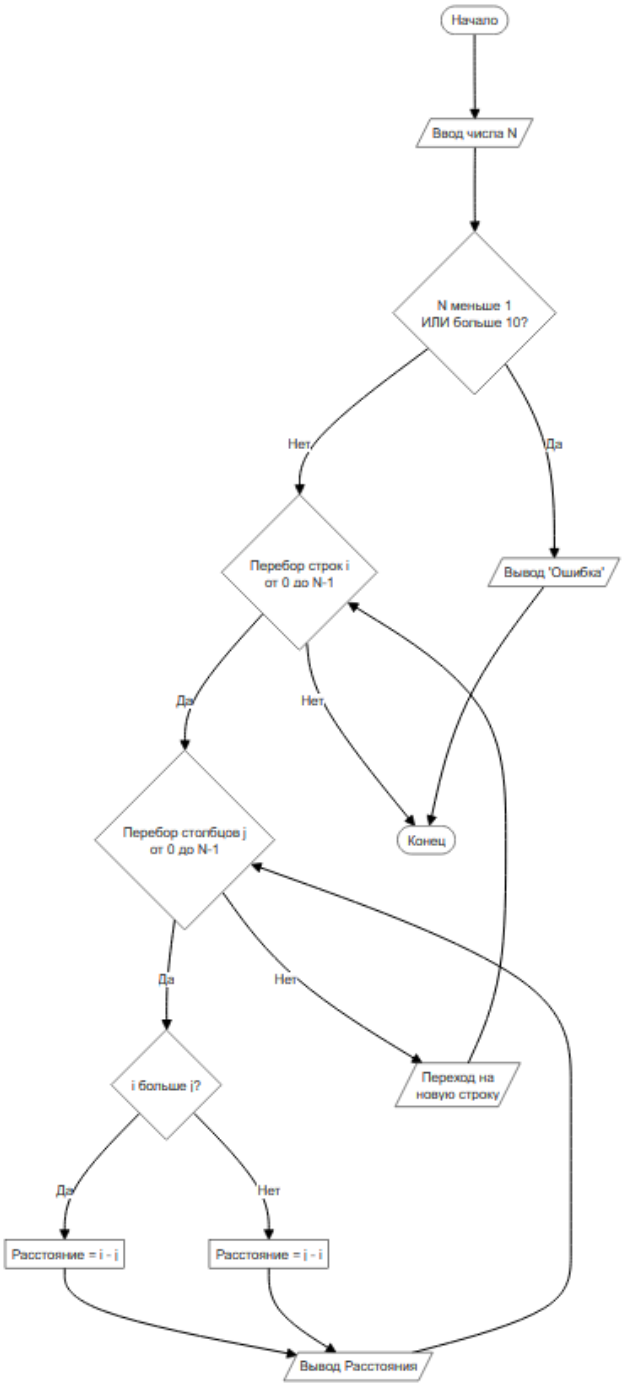


Рисунок 9 – Блок-схема программы для пятого задания

Текст программы для пятого задания представлен на рисунке 10.

```
n = int(input())

if n < 1 or n > 10:
    print("Ошибка")
else:
    for i in range(n):
        for j in range(n):
            if i > j:
                distance = i - j
            else:
                distance = j - i

            print(distance, end=" " if j < n - 1 else "")
        print()
```

Рисунок 10 – Текст программы для пятого задания

Программа формирует строки и столбцы таблицы, вычисляя разность координат по модулю через условие. Результаты выводятся в форматированном виде с пробелами между значениями, образуя двумерную таблицу.